



Morpho: Midnight whitelist enter gate audit

Auditors

MiloTruck, Lead Security Researcher

Saw-mon and Natalie, Lead Security Researcher

Om Parikh, Security Researcher

Report prepared by: Lucas Goiriz

September 16, 2026

Contents

1	About Spearbit	2
2	Introduction	2
3	Risk classification	2
3.1	Impact	2
3.2	Likelihood	2
3.3	Action required for severity levels	2
4	Executive Summary	3
4.1	Scope	3
5	Findings	4
5.1	Low Risk	4
5.1.1	Front-running signed whitelist toggles blocks settlement batches	4
5.2	Informational	4
5.2.1	Optional omission of <code>whitelister</code> from the EOA-only signed digest	4
5.2.2	Consider adding <code>name</code> and <code>version</code> to EIP712 domain	5
6	Appendix	6
6.1	<code>WhitelistEnterGate</code> role authority	6

1 About Spearbit

Spearbit is a decentralized network of expert security engineers offering reviews and other security related services to Web3 projects with the goal of creating a stronger ecosystem. Our network has experience on every part of the blockchain technology stack, including but not limited to protocol design, smart contracts and the Solidity compiler. Spearbit brings in untapped security talent by enabling expert freelance auditors seeking flexibility to work on interesting projects together.

Learn more about us at spearbit.com

2 Introduction

Morpho is a trustless and efficient lending primitive with permissionless market creation.

Disclaimer: This security review does not guarantee against a hack. It is a snapshot in time of Morpho: Midnight according to the specific commit. Any modifications to the code will require a new security review.

3 Risk classification

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

3.1 Impact

- High - leads to a loss of a significant portion (>10%) of assets in the protocol, or significant harm to a majority of users.
- Medium - global losses <10% or losses to only a subset of users, but still unacceptable.
- Low - losses will be annoying but bearable--applies to things like griefing attacks that can be easily repaired or even gas inefficiencies.

3.2 Likelihood

- High - almost certain to happen, easy to perform, or not easy but highly incentivized
- Medium - only conditionally possible or incentivized, but still relatively likely
- Low - requires stars to align, or little-to-no incentive

3.3 Action required for severity levels

- Critical - Must fix as soon as possible (if already deployed)
- High - Must fix (before deployment if not already deployed)
- Medium - Should fix
- Low - Could fix

4 Executive Summary

On Sep 10th, [Morpho](#) engaged with [Spearbit](#) to review the [midnight](#) protocol. In this period of time a total of **3** issues were found.

Summary

Project Name	Morpho
Repository	midnight
Commit	70607569
Type of Project	DeFi, Lending
Audit Timeline	Sep 10th

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	0	0	0
Low Risk	1	0	1
Gas Optimizations	0	0	0
Informational	2	1	1
Total	3	1	2

The Spearbit team reviewed Morpho's [midnight](#) holistically on commit hash [ecadee4b](#) and concluded that all findings were addressed and no new vulnerabilities were identified.

4.1 Scope

The security review had the following components in scope for [midnight](#) on commit hash [70607569](#):

```
# All files under the following directory  
src/periphery/whitelist-enter-gate
```

5 Findings

5.1 Low Risk

5.1.1 Front-running signed whitelist toggles blocks settlement batches

Severity: Low Risk.

Context: [WhitelistEnterGate.sol#L84-L87](#).

Description: Let n be the current nonce for a fixed `(creditSide, whitelister, account)`, with credit entry restricted. Consider a pending atomic batch containing two valid signed whitelist updates:

$\text{enable}(n) \rightarrow \text{take} \rightarrow \text{disable}(n + 1).$

The `signed payload` binds neither the caller nor the settlement. An observer can extract both signatures and execute `enable(n)` followed by `disable(n+1)` before the batch. The nonce becomes $n + 2$ and the account is disabled.

The batch's first call then reaches the `stale-nonce branch`: $n < n + 2$ holds, but the current status is `false` rather than the signed `true`. It reverts with `WhitelistedStatusChanged()`, preventing settlement. The attacker needs no whitelister role; both updates use the exposed signatures. Retrying with fresh signatures remains susceptible to the same ordering.

Recommendation: Bind temporary whitelist authorization to an executor that enforces the complete authorized settlement sequence atomically. Binding only to a permissionless bundler address is insufficient if arbitrary callers can use it to execute the extracted updates.

Keep the stale-status check: removing it leaves the account disabled, while reapplying stale grants would allow consumed signatures to override subsequent revocations.

Morpho: Acknowledged, thinking that it's niche enough and with "only" a liveness impact.

Spearbit: Acknowledged.

5.2 Informational

5.2.1 Optional omission of `whitelister` from the EOA-only signed digest

Severity: Informational.

Context: [IWhitelistEnterGate.sol#L7-L9](#), [WhitelistEnterGate.sol#L71-L73](#).

Description: `setIsWhitelistedWithSig` includes `whitelister` in the signed struct, then requires `ecrecover(digest, v, r, s) == whitelister`, a nonzero recovered address, and the whitelister role. Signature recovery already binds the authorization to the signer; nonces are scoped to that verified address.

For the current EOA verification scheme, omitting `whitelister` saves one word of struct encoding and hashing while retaining the signer and role checks. This is an optional optimization, not a correctness issue; retaining the field explicitly commits the signed payload to the authorizing identity.

Including `whitelister` may also be intentional future-proofing, for example for a later design supporting [ERC-1271](#). This is a possible rationale, not a confirmed plan; the current function does not call `ERC-1271`, and the integration design is unspecified.

`ERC-1271` asks a specified contract whether a signature is valid. Two wallets with the same owner may both accept the same digest and signature. If both have the whitelister role and matching nonces, omitting the wallet address can permit reuse across their nonce namespaces. The gate's domain separator identifies the gate, not the signing wallet. Binding `whitelister` distinguishes these authorizations.

Recommendation: Consider removal only for the current EOA-only scheme if the small saving is preferred over explicit identity binding. If the field is retained for future-proofing, document that rationale. If removed, update the struct encoding, EIP-712 type definition, typehash, signing helpers, and tests together; retain the signer, nonzero, role, and nonce checks. For ERC-1271 support, retain `whitelister` in the digest unless equivalent wallet-specific binding is enforced.

Morpho: Fixed in commit [ecadee4b](#).

Spearbit: [PR 1192](#) removes `whitelister` from the signed struct and instead derives the whitelister identity from the `ecrecover`-ed signer address, using it to look up the correct per-whitelister `nonce` from storage.

5.2.2 Consider adding `name` and `version` to EIP712 domain

Severity: Informational.

Context: *(No context files were provided by the reviewer).*

Description:

```
/// @dev keccak256("EIP712Domain(uint256 chainId,address verifyingContract)").
bytes32 constant EIP712_DOMAIN_TYPEHASH =
↳ 0x47e79534a245952e8b16893a336b85a3d9ea9fa8c573f3d803afb92a79469218;
```

The current EIP-712 domain does not include optional fields such as `name` and `version`.

Recommendation: Consider adding `name` and `version` to the EIP-712 domain for better UX and readability.

Morpho: Acknowledged:

- We usually don't put name as it's redundant with the address.
- And yes the version doesn't make sense since it's not upgradable.

Spearbit: Acknowledged.

6 Appendix

6.1 `WhitelistEnterGate` role authority

Let $s \in \{\text{credit}, \text{debt}\}$. Write R_s for the side's role setter, W_s for its set of whitelisters, and A_s for its set of whitelisted accounts. The authority graph is

$$R_s \longrightarrow R_s \longrightarrow W_s A_s.$$

Here $x \longrightarrow y$ means that x may change y : the first edge permits replacement of the role setter; the second permits adding or removing whitelisters. The repeated R_s denotes the same role. The multiple arrow means

$$\forall w \in W_s, \quad w \longrightarrow A_s : \quad A_s \leftarrow A_s \cup \{a\} \quad \text{or} \quad A_s \leftarrow A_s \setminus \{a\}.$$

Thus every whitelister independently controls every account's whitelist status. Whitelisters can override one another: for distinct $w_1, w_2 \in W_s$, w_1 may add an account and w_2 may remove it, or vice versa. The last executed update determines membership; no whitelister has priority or exclusive authority over an account. A valid signature from a current whitelister permits the same update through a relayer, subject to the nonce and deadline checks. A role setter can acquire this authority by adding itself to W_s .

The contract-level documentation names the roles but does not state this authority graph. In particular, removing a whitelister does not remove accounts it whitelisted. The two sides have separate authority graphs, although their role holders may coincide; an immutable open side admits all accounts regardless of A_s .